EFREIForge | Une start-up qui passe à l'échelle

Projet Java — LSI1 · 2026–2027

> **Votre mission :** faire évoluer l'application construite au fil des TP pour aider une start-up à organiser ses équipes. Vous avez découvert les outils ; à vous de choisir comment les mobiliser pour répondre à un besoin concret.

Le contexte

Alice et Alex ont lancé leur start-up. L'activité se développe, les recrutements s'enchaînent et l'application utilisée jusqu'ici pour gérer les collaborateurs ne suffit plus.

Les fondateurs souhaitent désormais **constituer des équipes**, savoir qui travaille dans chacune d'elles et conserver ces informations d'une exécution à l'autre.

Bonne nouvelle : vous ne partez pas de zéro. Vous disposez du travail réalisé au fil des TP, notamment du TP4 : collaborateurs, profils, règles métier, collections, gestion des erreurs et persistance.

Votre défi n'est pas d'ajouter le plus de fonctionnalités possibles. Il est de faire évoluer intelligemment une application existante.

Les besoins de la start-up

Organiser les équipes

L'application doit permettre de :

- Créer une équipe et lui donner un nom.
- Consulter les équipes existantes.
- Affecter un collaborateur à une équipe.
- Retirer un collaborateur d'une équipe.
- Afficher les collaborateurs d'une équipe.
- Renommer une équipe.
- Supprimer une équipe.

Quelques règles métier :

- Une équipe peut regrouper plusieurs collaborateurs.
- Un collaborateur appartient à **une seule équipe au maximum** ; il peut aussi n'appartenir à aucune équipe.
- Le nom d'une équipe est **obligatoire et unique**. La casse n'est pas prise en compte : « Alpha » et « alpha » désignent la même équipe. Ces règles s'appliquent aussi au renommage (une équipe peut modifier la casse de son propre nom).
- **Affecter** un collaborateur qui a déjà une équipe est **refusé**, y compris vers la même équipe : il n'y a pas de déplacement implicite.

- **Retirer** un collaborateur d'une équipe n'est possible que s'il appartient bien à cette équipe. Sinon, l'opération est refusée (cela couvre le collaborateur sans équipe comme celui d'une autre équipe). Votre interface demande donc **à la fois** le collaborateur et l'équipe concernée.
- Une équipe ne peut être **supprimée** que si elle ne contient plus aucun collaborateur.
- Les opérations incohérentes (équipe ou collaborateur inexistant, par exemple) doivent être traitées proprement. **Une opération refusée ne modifie jamais les données.**
- Les équipes **et leurs affectations** doivent être retrouvées après redémarrage de l'application.

La start-up souhaite conserver les fonctionnalités utiles déjà présentes dans votre application. Il ne s'agit pas de tout réécrire.

Repérer les collaborateurs disponibles

Les fondateurs souhaitent obtenir **la liste des collaborateurs sans équipe**, afin de faciliter les prochaines affectations.

À titre d'illustration (la présentation reste libre) :

```
Collaborateurs sans équipe (3)
  C003  Chloé Bernard   Testeur
  C005  Hugo Petit      Testeur
  C006  Lina Moreau     Programmeur (Kotlin)
```

Si tous les collaborateurs ont une équipe, un message clair doit l'indiquer.

Garder une application utilisable

Un membre de la start-up doit pouvoir effectuer ces opérations depuis l'interface console. Les saisies incorrectes et opérations impossibles doivent produire un comportement compréhensible et cohérent.

Votre liberté de conception

Le sujet décrit les besoins, pas la liste des classes ou des méthodes à créer. Vous choisissez l'organisation du code, les responsabilités, les collections, les requêtes et l'interface console.

Le projet doit néanmoins :

- Repartir de **votre application issue du TP4** et réutiliser au maximum ce qui fonctionne.
- Mobiliser les principes de programmation orientée objet étudiés.
- Utiliser **JPA pour persister les équipes et les affectations**, en réutilisant les mécanismes du TP4.
- Préserver la persistance des collaborateurs déjà mise en place.
- Gérer les erreurs significatives.
- Rester un projet Maven exécutable.

Il n'est pas demandé de complexifier les mappings JPA. Une association simple, unidirectionnelle, suffit. Aucune association bidirectionnelle, relation `@ManyToMany`, nouvelle stratégie d'héritage ou configuration avancée n'est exigée.

Un coup de pouce pour la persistance

Pour vous permettre de vous concentrer sur les fonctionnalités métier, voici le **mapping JPA de départ** de la nouvelle association. Vous n'avez pas à le redécouvrir : votre travail consiste à l'intégrer et à l'exploiter correctement.

Nouvelle entité `Equipe` (squelette à compléter) :

```
@Entity
@Table(name = "equipe")
public class Equipe {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;

    protected Equipe() {
        // exigé par JPA, comme pour vos autres entités
    }

    // Constructeur public, accesseurs et autres éléments utiles à compléter.
}
```

Dans votre entité `Collaborateur` **existante**, ajoutez l'association :

```
@ManyToOne
@JoinColumn(name = "equipe_id")
private Equipe equipe;
```

Ce mapping permet à plusieurs collaborateurs d'appartenir à une même équipe, tandis qu'un collaborateur ne référence qu'une seule équipe au maximum. **L'association est unidirectionnelle** : aucune collection de collaborateurs n'est nécessaire dans `Equipe`. L'absence d'équipe reste possible.

Attention, quatre points à ne pas oublier :

- **Déclarez** `Equipe` **dans** `persistence.xml`, avec une ligne `<class>` comme pour vos autres entités. Sinon, JPA répondra « Unknown entity ».
- Une équipe doit être **enregistrée avant** de pouvoir être associée à un collaborateur.
- L'association ne comporte pas de `cascade` : pour affecter un collaborateur, chargez l'équipe **et** le collaborateur avec le même `EntityManager`, dans la même transaction. La modification du collaborateur est alors synchronisée comme l'augmentation de salaire du TP4.

- **Alignez votre script SQL sur votre mapping** : mêmes noms de tables et de colonnes (ici `equipe`, `nom`, `equipe_id`). L'annotation `@Table` fixe le nom de la table sans ambiguïté ; selon le système, MariaDB distingue les majuscules des minuscules dans les noms de tables.

La nouvelle colonne `equipe_id` est facultative : Hibernate l'ajoutera à votre table sans toucher aux collaborateurs déjà enregistrés. Adaptez les imports et les types d'identifiants à votre projet existant.

Dans ce squelette, `nom` n'a **aucune contrainte** : Hibernate le créera facultatif et non unique. Le nom obligatoire et unique (voir les règles métier) est donc à garantir **dans votre code** ; la base ne le fera pas à votre place, sauf si vous y ajoutez vous-même une contrainte.

Le mapping est fourni ; **les opérations métier, les accès aux données, les requêtes et les contrôles de cohérence restent à concevoir et à développer par votre binôme**.

> **TP4 incomplet ?** Reprenez ses parties fonctionnelles et concentrez-vous sur ce qui est nécessaire au projet. Il n'est pas demandé de terminer rétrospectivement toutes les missions du TP4. Indiquez dans votre journal les limites qui ont un impact sur votre livraison.

Pour aller plus loin [facultatif]

Si le socle fonctionne et que vous disposez encore de temps, vous pouvez :

- Permettre le transfert direct d'un collaborateur d'une équipe vers une autre, en **une seule opération** qui réussit ou échoue entièrement (aucun collaborateur ne doit se retrouver sans équipe si le transfert échoue).
- Afficher une vue d'ensemble des effectifs par équipe et du nombre de collaborateurs sans équipe.

Ces approfondissements sont **facultatifs**. Leur absence n'est pas pénalisée ; leur réalisation maîtrisée peut donner lieu à un bonus.

Livraison : un projet, un journal

Le projet est réalisé en **binôme**. Déposez sur Moodle, **une seule fois par binôme**, **une archive ZIP du projet Maven**, avec le code source, un seul fichier `JOURNAL.md` à la racine et un script SQL (par exemple dans un dossier `sql/`). Évitez les répertoires de compilation et les fichiers temporaires de l'IDE.

Le journal doit rester **court et utile (environ deux pages)**. Il comporte :

1. **Démarche :** organisation du travail et éléments du TP4 réutilisés.
2. **Deux décisions techniques :** choix réalisés et raisons de ces choix.
3. **État du projet :** ce qui fonctionne, ce qui reste incomplet et les difficultés significatives, avec le tableau de recette complété.
4. **README — Lancer et essayer l'application :** prérequis, procédure pour préparer la base à partir du script SQL, démarrage de l'application et quelques manipulations permettant de vérifier le résultat.
5. **Contribution de chacun :** en quelques lignes, ce que chaque membre du binôme a réalisé.

Aucun `README.md` séparé n'est demandé.

Le script SQL et le jeu de données

L'enseignant doit pouvoir lancer votre application à partir d'une base vide. Vous fournissez donc un script SQL qui :

- Crée la base `efreiforge` si elle n'existe pas, puis les tables nécessaires ;
- Insère un **jeu de données** permettant d'essayer l'application : au moins 6 collaborateurs (programmeurs et testeurs, avec leurs adresses), 2 équipes, des affectations, et au moins 2 collaborateurs sans équipe.

Votre `persistence.xml` doit viser la configuration par défaut de XAMPP (`localhost:3306`, utilisateur `root`, sans mot de passe) et la base `efreiforge`. Le journal indique comment importer le script (par exemple avec l'onglet *Importer* de phpMyAdmin). Le script doit pouvoir être **rejoué** sans erreur : l'enseignant l'exécutera peut-être plusieurs fois.

Le tableau de recette

Reproduisez ce tableau dans la section « État du projet » du journal et indiquez, pour chaque scénario, le résultat **réellement constaté** (OK, KO ou partiel), avec un commentaire si nécessaire. Un KO honnête vaut mieux qu'un OK non vérifié. Vous pouvez ajouter vos propres scénarios.

N°	Scénario	Résultat attendu	Constaté
1	Créer l'équipe « Alpha », puis « alpha », puis une équipe sans nom (vide ou espaces).	La première est créée ; les deux autres sont refusées.	
2	Affecter à « Alpha » un collaborateur sans équipe, puis afficher « Alpha ».	Le collaborateur apparaît dans l'équipe.	
3	Affecter ce même collaborateur à une autre équipe, puis de nouveau à « Alpha ».	Deux refus ; il reste dans « Alpha ».	
4	Retirer d'« Alpha » un collaborateur qui n'en fait pas partie (d'abord un sans équipe, puis un membre d'une autre équipe).	Deux refus, aucune donnée modifiée.	
5	Afficher les collaborateurs sans équipe.	Le collaborateur affecté n'y figure pas ; les autres y sont.	
6	Arrêter puis relancer l'application, consulter équipes et affectations.	Tout est retrouvé à l'identique.	
7	Renommer une équipe : changer seulement la casse de son nom, puis prendre le nom d'une autre équipe. Supprimer une équipe non vide, puis une équipe vide.	Casse seule : acceptée ; nom déjà pris : refusé. Suppression refusée tant que l'équipe contient des collaborateurs, acceptée ensuite.	
8	Utiliser une équipe ou un collaborateur inexistant ; saisir du texte quand un nombre est attendu.	Message clair, l'application ne plante pas.	

Avant de déposer : testez votre livraison comme le fera l'enseignant. Dézippez l'archive dans un dossier neuf, importez le script dans une base vide, lancez l'application, puis rejouez la recette.

Évaluation

Seront particulièrement pris en compte :

- **Le fonctionnement :** besoins couverts, respect des règles métier, cohérence des opérations et persistance effective.
- **La conception :** objets, collections et répartition des responsabilités.
- **La maîtrise technique :** réutilisation de JPA, gestion des erreurs, lisibilité et simplicité du code.
- **La qualité de vos livrables :** projet exécutable à partir du script SQL, `JOURNAL.md` exploitable.
- **Votre autonomie.**

Les approfondissements de la section "Pour aller plus loin" peuvent donner lieu à un bonus (jusqu'à 1 point). La note est attribuée au binôme ; la section « Contribution de chacun » du journal peut éclairer un éventuel déséquilibre.

Aucune soutenance n'est prévue. Les deux séances sont consacrées au développement et à l'amélioration des livrables.

Un projet modeste, cohérent et fiable vaut mieux qu'un projet ambitieux mais fragile.

Calendrier [LSI1]

- **PRJ 1 :** jeudi 15 octobre 2026.
- **PRJ 2 :** vendredi 16 octobre 2026.
- **Dépôt sur Moodle : vendredi 23 octobre 2026 à 23h59.**

Le délai après la dernière séance sert à finaliser et sécuriser la livraison, non à imposer de nouvelles fonctionnalités.

Alice et Alex ont besoin d'une application qui les aide à travailler, pas d'une démonstration de toutes les fonctionnalités possibles de Java. À vous de jouer !